For more info on these scenarios and owning types, see the "More about Canvas.Left" section of Custom attached properties.

Attached properties in code

Attached properties don't have the typical property wrappers for easy get and set access like other dependency properties do. This is because the attached property is not necessarily part of the code-centered object model for instances where the property is set. (It is permissible, though uncommon, to define a property that is both an attached property that other types can set on themselves, and that also has a conventional property usage on the owning type.)

There are two ways to set an attached property in code: use the property-system APIs, or use the XAML pattern accessors. These techniques are pretty much equivalent in terms of their end result, so which one to use is mostly a matter of coding style.

Using the property system

Attached properties for the Windows Runtime are implemented as dependency properties, so that the values can be stored in the shared dependency-property store by the property system. Therefore attached properties expose a dependency property identifier on the owning class.

To set an attached property in code, you call the **SetValue** method, and pass the **DependencyProperty** field that serves as the identifier for that attached property. (You also pass the value to set.)

To get the value of an attached property in code, you call the **GetValue** method, again passing the **DependencyProperty** field that serves as the identifier.

Using the XAML accessor pattern

A XAML processor must be able to set attached property values when XAML is parsed into an object tree. The owner type of the attached property must implement dedicated accessor methods named in the form **Get***PropertyName* and **Set***PropertyName*. These dedicated accessor methods are also one way to get or set the attached property in code. From a code perspective, an attached property is similar to a backing field that has method accessors instead of property accessors, and that backing field can exist on any object rather than having to be specifically defined.

The next example shows how you can set an attached property in code via the XAML accessor API. In this example, `myCheckBox` is an instance of the **CheckBox** class. The last line is the code that actually sets the value; the lines before that just establish the instances and their parent-child relationship. The uncommented last line is the syntax if you use the property system. The commented last line is the syntax if you use the XAML accessor pattern.

C#

```
    Canvas myC = new Canvas();
    CheckBox myCheckBox = new CheckBox();
    myCheckBox.Content = "Hello";
    myC.Children.Add(myCheckBox);
    myCheckBox.SetValue(Canvas.TopProperty,75);
    //Canvas.SetTop(myCheckBox, 75);
```

C++/WinRT

```
Canvas myC;
CheckBox myCheckBox;
myCheckBox.Content(winrt::box_value(L"Hello"));
myC.Children().Append(myCheckBox);
myCheckBox.SetValue(Canvas::TopProperty(), winrt::box_value(75));
// Canvas::SetTop(myCheckBox, 75);
```

C++

```
    Canvas^ myC = ref new Canvas();
    CheckBox^ myCheckBox = ref new CheckBox();
    myCheckBox->Content="Hello";
    myC->Children->Append(myCheckBox);
    myCheckBox->SetValue(Canvas::TopProperty,75);
    // Canvas::SetTop(myCheckBox, 75);
```

Custom attached properties

For code examples of how to define custom attached properties, and more info about the scenarios for using an attached property, see Custom attached properties.

Special syntax for attached property references

The dot in an attached property name is a key part of the identification pattern. Sometimes there are ambiguities when a syntax or situation treats the dot as having some other meaning. For example, a dot is treated as an object-model traversal for a

binding path. In most cases involving such ambiguity, there is a special syntax for an attached property that enables the inner dot still to be parsed as the *owner.property* separator of an attached property.

- To specify an attached property as part of a target path for an animation, enclose the attached property name in parentheses ("()")—for example, "(Canvas.Left)". For more info, see Property-path syntax.

> ⚠ **Warning**
>
> An existing limitation of the Windows Runtime XAML implementation is that you cannot animate a custom attached property.

- To specify an attached property as the target property for a resource reference from a resource file to **x:Uid**, use a special syntax that injects a code-style, fully qualified **using:** declaration inside square brackets ("[]"), to create a deliberate scope break. For example, assuming there exists an element `<TextBlock x:Uid="Title" />`, the resource key in the resource file that targets the **Canvas.Top** value on that instance is "Title.[using:Windows.UI.Xaml.Controls]Canvas.Top". For more info on resource files and XAML, see Quickstart: Translating UI resources.

Related topics

- Custom attached properties
- Dependency properties overview
- Define layouts with XAML
- Quickstart: Translating UI resources
- **SetValue**
- **GetValue**

Custom attached properties

Article • 10/20/2022

An *attached property* is a XAML concept. Attached properties are typically defined as a specialized form of dependency property. This topic explains how to implement an attached property as a dependency property and how to define the accessor convention that is necessary for your attached property to be usable in XAML.

Prerequisites

We assume that you understand dependency properties from the perspective of a consumer of existing dependency properties, and that you have read the Dependency properties overview. You should also have read Attached properties overview. To follow the examples in this topic, you should also understand XAML and know how to write a basic Windows Runtime app using C++, C#, or Visual Basic.

Scenarios for attached properties

You might create an attached property when there is a reason to have a property-setting mechanism available for classes other than the defining class. The most common scenarios for this are layout and services support. Examples of existing layout properties are **Canvas.ZIndex** and **Canvas.Top**. In a layout scenario, elements that exist as child elements to layout-controlling elements can express layout requirements to their parent elements individually, each setting a property value that the parent defines as an attached property. An example of the services-support scenario in the Windows Runtime API is set of the attached properties of **ScrollViewer**, such as **ScrollViewer.IsZoomChainingEnabled**.

> ⚠ **Warning**
>
> An existing limitation of the Windows Runtime XAML implementation is that you cannot animate your custom attached property.

Registering a custom attached property

If you are defining the attached property strictly for use on other types, the class where the property is registered does not have to derive from **DependencyObject**. But you do need to have the target parameter for accessors use **DependencyObject** if you follow

the typical model of having your attached property also be a dependency property, so that you can use the backing property store.

Define your attached property as a dependency property by declaring a **public static readonly** property of type DependencyProperty. You define this property by using the return value of the RegisterAttached method. The property name must match the attached property name you specify as the **RegisterAttached** *name* parameter, with the string "Property" added to the end. This is the established convention for naming the identifiers of dependency properties in relation to the properties that they represent.

The main area where defining a custom attached property differs from a custom dependency property is in how you define the accessors or wrappers. Instead of the using the wrapper technique described in Custom dependency properties, you must also provide static **Get***PropertyName* and **Set***PropertyName* methods as accessors for the attached property. The accessors are used mostly by the XAML parser, although any other caller can also use them to set values in non-XAML scenarios.

> **Important**
>
> If you don't define the accessors correctly, the XAML processor can't access your attached property and anyone who tries to use it will probably get a XAML parser error. Also, design and coding tools often rely on the "*Property" conventions for naming identifiers when they encounter a custom dependency property in a referenced assembly.

Accessors

The signature for the **Get***PropertyName* accessor must be this.

`public static` *valueType* **Get***PropertyName* `(DependencyObject target)`

For Microsoft Visual Basic, it is this.

`Public Shared Function Get`*PropertyName*`(ByVal target As DependencyObject) As` *valueType* `)`

The *target* object can be of a more specific type in your implementation, but must derive from DependencyObject. The *valueType* return value can also be of a more specific type in your implementation. The basic **Object** type is acceptable, but often you'll want your attached property to enforce type safety. The use of typing in the getter and setter signatures is a recommended type-safety technique.

The signature for the **Set***PropertyName* accessor must be this.

`public static void Set`*PropertyName*`(DependencyObject target ,`*valueType*` value)`

For Visual Basic, it is this.

`Public Shared Sub Set`*PropertyName*`(ByVal target As DependencyObject, ByVal value As`*valueType*`)`

The *target* object can be of a more specific type in your implementation, but must derive from **DependencyObject**. The *value* object and its *valueType* can be of a more specific type in your implementation. Remember that the value for this method is the input that comes from the XAML processor when it encounters your attached property in markup. There must be type conversion or existing markup extension support for the type you use, so that the appropriate type can be created from an attribute value (which is ultimately just a string). The basic **Object** type is acceptable, but often you'll want further type safety. To accomplish that, put type enforcement in the accessors.

> **Note**
>
> It's also possible to define an attached property where the intended usage is through property element syntax. In that case you don't need type conversion for the values, but you do need to assure that the values you intend can be constructed in XAML. **VisualStateManager.VisualStateGroups** is an example of an existing attached property that only supports property element usage.

Code example

This example shows the dependency property registration (using the **RegisterAttached** method), as well as the **Get** and **Set** accessors, for a custom attached property. In the example, the attached property name is `IsMovable`. Therefore, the accessors must be named `GetIsMovable` and `SetIsMovable`. The owner of the attached property is a service class named `GameService` that doesn't have a UI of its own; its purpose is only to provide the attached property services when the **GameService.IsMovable** attached property is used.

Defining the attached property in C++/CX is a bit more complex. You have to decide how to factor between the header and code file. Also, you should expose the identifier as a property with only a **get** accessor, for reasons discussed in Custom dependency properties. In C++/CX you must define this property-field relationship explicitly rather than relying on .NET **readonly** keywording and implicit backing of simple properties. You

also need to perform the registration of the attached property within a helper function that only gets run once, when the app first starts but before any XAML pages that need the attached property are loaded. The typical place to call your property registration helper functions for any and all dependency or attached properties is from within the **App** / **Application** constructor in the code for your app.xaml file.

C#

```
public class GameService : DependencyObject
{
    public static readonly DependencyProperty IsMovableProperty =
    DependencyProperty.RegisterAttached(
      "IsMovable",
      typeof(Boolean),
      typeof(GameService),
      new PropertyMetadata(false)
    );
    public static void SetIsMovable(UIElement element, Boolean value)
    {
        element.SetValue(IsMovableProperty, value);
    }
    public static Boolean GetIsMovable(UIElement element)
    {
        return (Boolean)element.GetValue(IsMovableProperty);
    }
}
```

Setting your custom attached property from XAML markup

After you have defined your attached property and included its support members as part of a custom type, you must then make the definitions available for XAML usage. To do this, you must map a XAML namespace that will reference the code namespace that contains the relevant class. In cases where you have defined the attached property as part of a library, you must include that library as part of the app package for the app.

An XML namespace mapping for XAML is typically placed in the root element of a XAML page. For example, for the class named `GameService` in the namespace `UserAndCustomControls` that contains the attached property definitions shown in preceding snippets, the mapping might look like this.

XAML

```
<UserControl
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
    xmlns:uc="using:UserAndCustomControls"
    ... >
```

Using the mapping, you can set your `GameService.IsMovable` attached property on any element that matches your target definition, including an existing type that Windows Runtime defines.

XAML

```
<Image uc:GameService.IsMovable="True" .../>
```

If you are setting the property on an element that is also within the same mapped XML namespace, you still must include the prefix on the attached property name. This is because the prefix qualifies the owner type. The attached property's attribute cannot be assumed to be within the same XML namespace as the element where the attribute is included, even though, by normal XML rules, attributes can inherit namespace from elements. For example, if you are setting `GameService.IsMovable` on a custom type of `ImageWithLabelControl` (definition not shown), and even if both were defined in the same code namespace mapped to same prefix, the XAML would still be this.

XAML

```
<uc:ImageWithLabelControl uc:GameService.IsMovable="True" .../>
```

> **Note**
>
> If you are writing a XAML UI with C++/CX, then you must include the header for the custom type that defines the attached property, any time that a XAML page uses that type. Each XAML page has an associated code-behind header (.xaml.h). This is where you should include (using **#include**) the header for the definition of the attached property's owner type.

Setting your custom attached property imperatively

You can also access a custom attached property from imperative code. The code below shows how.

XAML

```
<Image x:Name="gameServiceImage"/>
```

C++/WinRT

```
// MainPage.h
...
#include "GameService.h"
...

// MainPage.cpp
...
MainPage::MainPage()
{
    InitializeComponent();

    GameService::SetIsMovable(gameServiceImage(), true);
}
...
```

Value type of a custom attached property

The type that is used as the value type of a custom attached property affects the usage, the definition, or both the usage and definition. The attached property's value type is declared in several places: in the signatures of both the **Get** and **Set** accessor methods, and also as the *propertyType* parameter of the **RegisterAttached** call.

The most common value type for attached properties (custom or otherwise) is a simple string. This is because attached properties are generally intended for XAML attribute usage, and using a string as the value type keeps the properties lightweight. Other primitives that have native conversion to string methods, such as integer, double, or an enumeration value, are also common as value types for attached properties. You can use other value types—ones that don't support native string conversion—as the attached property value. However, this entails making a choice about either the usage or the implementation:

- You can leave the attached property as it is, but the attached property can support usage only where the attached property is a property element, and the value is declared as an object element. In this case, the property type does have to support XAML usage as an object element. For existing Windows Runtime reference classes, check the XAML syntax to make sure that the type supports XAML object element usage.
- You can leave the attached property as it is, but use it only in an attribute usage through a XAML reference technique such as a **Binding** or **StaticResource** that can

be expressed as a string.

More about the Canvas.Left example

In earlier examples of attached property usages we showed different ways to set the Canvas.Left attached property. But what does that change about how a Canvas interacts with your object, and when does that happen? We'll examine this particular example further, because if you implement an attached property, it's interesting to see what else a typical attached property owner class intends to do with its attached property values if it finds them on other objects.

The main function of a Canvas is to be an absolute-positioned layout container in UI. The children of a **Canvas** are stored in a base-class defined property Children. Of all the panels **Canvas** is the only one that uses absolute positioning. It would've bloated the object model of the common UIElement type to add properties that might only be of concern to **Canvas** and those particular **UIElement** cases where they are child elements of a **UIElement**. Defining the layout control properties of a **Canvas** to be attached properties that any **UIElement** can use keeps the object model cleaner.

In order to be a practical panel, Canvas has behavior that overrides the framework-level Measure and Arrange methods. This is where **Canvas** actually checks for attached property values on its children. Part of both the **Measure** and **Arrange** patterns is a loop that iterates over any content, and a panel has the Children property that makes it explicit what's supposed to be considered the child of a panel. So the **Canvas** layout behavior iterates through these children, and makes static Canvas.GetLeft and Canvas.GetTop calls on each child to see whether those attached properties contain a non-default value (default is 0). These values are then used to absolutely position each child in the **Canvas** available layout space according to the specific values provided by each child, and committed using **Arrange**.

The code looks something like this pseudocode.

syntax

```
protected override Size ArrangeOverride(Size finalSize)
{
    foreach (UIElement child in Children)
    {
        double x = (double) Canvas.GetLeft(child);
        double y = (double) Canvas.GetTop(child);
        child.Arrange(new Rect(new Point(x, y), child.DesiredSize));
    }
    return base.ArrangeOverride(finalSize);
```

```
    // real Canvas has more sophisticated sizing
}
```

> **Note**
>
> For more info on how panels work, see **XAML custom panels overview**.

Related topics

- **RegisterAttached**
- Attached properties overview
- Custom dependency properties
- XAML overview

Events and routed events overview

Article • 10/20/2022

Important APIs

- UIElement
- RoutedEventArgs

We describe the programming concept of events in a Windows Runtime app, when using C#, Visual Basic or Visual C++ component extensions (C++/CX) as your programming language, and XAML for your UI definition. You can assign handlers for events as part of the declarations for UI elements in XAML, or you can add the handlers in code. Windows Runtime supports *routed events*: certain input events and data events can be handled by objects beyond the object that fired the event. Routed events are useful when you define control templates, or use pages or layout containers.

Events as a programming concept

Generally speaking, event concepts when programming a Windows Runtime app are similar to the event model in most popular programming languages. If you know how to work with Microsoft .NET or C++ events already, you have a head start. But you don't need to know that much about event model concepts to perform some basic tasks, such as attaching handlers.

When you use C#, Visual Basic or C++/CX as your programming language, the UI is defined in markup (XAML). In XAML markup syntax, some of the principles of connecting events between markup elements and runtime code entities are similar to other Web technologies, such as ASP.NET, or HTML5.

Note The code that provides the runtime logic for a XAML-defined UI is often referred to as *code-behind* or the code-behind file. In the Microsoft Visual Studio solution views, this relationship is shown graphically, with the code-behind file being a dependent and nested file versus the XAML page it refers to.

Button.Click: an introduction to events and XAML

One of the most common programming tasks for a Windows Runtime app is to capture user input to the UI. For example, your UI might have a button that the user must click to submit info or to change state.

You define the UI for your Windows Runtime app by generating XAML. This XAML is usually the output from a design surface in Visual Studio. You can also write the XAML in a plain-text editor or a third-party XAML editor. While generating that XAML, you can wire event handlers for individual UI elements at the same time that you define all the other XAML attributes that establish property values of that UI element.

To wire the events in XAML, you specify the string-form name of the handler method that you've already defined or will define later in your code-behind. For example, this XAML defines a **Button** object with other properties (x:Name attribute, **Content**) assigned as attributes, and wires a handler for the button's **Click** event by referencing a method named `ShowUpdatesButton_Click`:

XAML

```
<Button x:Name="showUpdatesButton"
  Content="{Binding ShowUpdatesText}"
  Click="ShowUpdatesButton_Click"/>
```

Tip *Event wiring* is a programming term. It refers to the process or code whereby you indicate that occurrences of an event should invoke a named handler method. In most procedural code models, event wiring is implicit or explicit "AddHandler" code that names both the event and method, and usually involves a target object instance. In XAML, the "AddHandler" is implicit, and event wiring consists entirely of naming the event as the attribute name of an object element, and naming the handler as that attribute's value.

You write the actual handler in the programming language that you're using for all your app's code and code-behind. With the attribute `Click="ShowUpdatesButton_Click"`, you have created a contract that when the XAML is markup-compiled and parsed, both the XAML markup compile step in your IDE's build action and the eventual XAML parse when the app loads can find a method named `ShowUpdatesButton_Click` as part of the app's code. `ShowUpdatesButton_Click` must be a method that implements a compatible method signature (based on a delegate) for any handler of the **Click** event. For example, this code defines the `ShowUpdatesButton_Click` handler.

C#

```
private void ShowUpdatesButton_Click (object sender, RoutedEventArgs e)
{
    Button b = sender as Button;
    //more logic to do here...
}
```

VB

```
Private Sub ShowUpdatesButton_Click(ByVal sender As Object, ByVal e As
RoutedEventArgs)
    Dim b As Button = CType(sender, Button)
    '  more logic to do here...
End Sub
```

C++/WinRT

```
void
winrt::MyNamespace::implementation::BlankPage::ShowUpdatesButton_Click(Windo
ws::Foundation::IInspectable const& sender,
Windows::UI::Xaml::RoutedEventArgs const& e)
{
    auto b{ sender.as<Windows::UI::Xaml::Controls::Button>() };
    // More logic to do here.
}
```

C++

```
void MyNamespace::BlankPage::ShowUpdatesButton_Click(Platform::Object^
sender, Windows::UI::Xaml::RoutedEventArgs^ e)
{
    Button^ b = (Button^) sender;
    //more logic to do here...
}
```

In this example, the `ShowUpdatesButton_Click` method is based on the RoutedEventHandler delegate. You'd know that this is the delegate to use because you'll see that delegate named in the syntax for the Click method.

Tip Visual Studio provides a convenient way to name the event handler and define the handler method while you're editing XAML. When you provide the attribute name of the event in the XAML text editor, wait a moment until a Microsoft IntelliSense list displays. If you click **<New Event Handler>** from the list, Microsoft Visual Studio will suggest a method name based on the element's **x:Name** (or type name), the event name, and a numeric suffix. You can then right-click the selected event handler name and click **Navigate to Event Handler**. This will navigate directly to the newly inserted event handler definition, as seen in the code editor view of your code-behind file for the XAML page. The event handler already has the correct signature, including the *sender* parameter and the event data class that the event uses. Also, if a handler method with the correct signature already exists in your code-behind, that method's name appears in the auto-complete drop-down along with the **<New Event Handler>** option. You can also press the Tab key as a shortcut instead of clicking the IntelliSense list items.

Defining an event handler

For objects that are UI elements and declared in XAML, event handler code is defined in the partial class that serves as the code-behind for a XAML page. Event handlers are methods that you write as part of the partial class that is associated with your XAML. These event handlers are based on the delegates that a particular event uses. Your event handler methods can be public or private. Private access works because the handler and instance created by the XAML are ultimately joined by code generation. In general, we recommend that you make your event handler methods private in the class.

Note Event handlers for C++ don't get defined in partial classes, they are declared in the header as a private class member. The build actions for a C++ project take care of generating code that supports the XAML type system and code-behind model for C++.

The *sender* parameter and event data

The handler you write for the event can access two values that are available as input for each case where your handler is invoked. The first such value is *sender*, which is a reference to the object where the handler is attached. The *sender* parameter is typed as the base **Object** type. A common technique is to cast *sender* to a more precise type. This technique is useful if you expect to check or change state on the *sender* object itself. Based on your own app design, you usually know a type that is safe to cast *sender* to, based on where the handler is attached or other design specifics.

The second value is event data, which generally appears in syntax definitions as the *e* parameter. You can discover which properties for event data are available by looking at the *e* parameter of the delegate that is assigned for the specific event you are handling, and then using IntelliSense or Object Browser in Visual Studio. Or you can use the Windows Runtime reference documentation.

For some events, the event data's specific property values are as important as knowing that the event occurred. This is especially true of the input events. For pointer events, the position of the pointer when the event occurred might be important. For keyboard events, all possible key presses fire a **KeyDown** and **KeyUp** event. To determine which key a user pressed, you must access the **KeyRoutedEventArgs** that is available to the event handler. For more info about handling input events, see Keyboard interactions and Handle pointer input. Input events and input scenarios often have additional considerations that are not covered in this topic, such as pointer capture for pointer events, and modifier keys and platform key codes for keyboard events.

Event handlers that use the async pattern